

[← Go Back](#)

Hardware

- Arduino UNO R4 WiFi CAN Bus
- Arduino UNO R4 WiFi User Manual
- Arduino UNO R4 WiFi
- Arduino Cloud Setup Guide
- Arduino UNO R4 WiFi Digital-to-Analog Converter (DAC)
- Debugging the Arduino UNO R4 WiFi
- Arduino UNO R4 WiFi EEPROM
- UNO R4 WiFi Custom Firmware Upload to ESP32 (Advanced)
- Using the Arduino UNO R4 WiFi LED Matrix**
- Arduino UNO R4 WiFi OPAMP
- Arduino UNO R4 WiFi Qwiic Connector
- Getting Started with UNO R4 WiFi
- Arduino UNO R4 WiFi Real-Time Clock
- UNO R4 WiFi Stack Trace Debug Runtime Errors
- UNO R4 Capacitive-Touch Tutorial
- Arduino UNO R4 WiFi USB HID
- Arduino UNO R4 WiFi VRTC & OFF Pins
- UNO R4 WiFi Network Examples

Home / Hardware / UNO R4 WiFi / **Using the Arduino UNO R4 WiFi LED Matrix**

Using the Arduino UNO R4 WiFi LED Matrix

Get off the ground with the Arduino UNO R4 WiFi built-in LED matrix. Learn the different techniques for controlling it, create animations, graphics or even games.

Jacob Hylén
Author & Tom Igoe
Last revision · 10/07/2025

The **Arduino UNO R4 WiFi** comes with a built in 12x8 LED Matrix, that is available to be programmed to display graphics, animations, act as an interface, or even play games on.

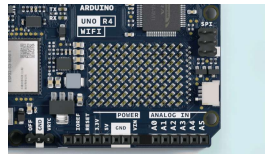
Goals

The matrix and its API are developed to be programmed in a few different ways, each suited for different applications. This guide will walk you through the basic concepts for programming the LED matrix, and get you started with creating your own animations, while highlighting two different ways of handling the LEDs to create animations and images.

ON THIS PAGE

- Goals**
- Hardware & Software Needed
- Initializing Matrix
- How to Write a Frame
- Testing It Out
- Resources
- Scrolling Text Example
- Frame Gallery
- Animation Generation
- API
- Members +

decide what method fits your needs best!



The UNO R4 WiFi LED
Matrix

Hardware & Software Needed

[Arduino UNO R4 WiFi](#)

[UNO R4 Board Package](#)
(latest version)

[Arduino IDE](#)

Initializing Matrix

To use the LED Matrix library, there are a few things that need to be added to your sketch to get off the ground.

First, include the library at the top of your sketch, like this:

```
1 #include "Arduino_LED_
```

Then, you'll need to create a LED Matrix object in your sketch, by adding the following line directly underneath the first one:

```
1  ArduinoLEDMatrix matrix;
```

And then lastly, start the LED Matrix by adding this line in

`void setup()`:

```
1  matrix.begin();
```

The entire thing should look like this;

```
1  #include "Arduino_LED_Matrix.h";
2
3  ArduinoLEDMatrix matrix;
4
5  void setup() {
6      Serial.begin(115200);
7      matrix.begin();
8  }
```

How to Write a Frame

The LED Matrix library for the UNO R4 WiFi works on the principle of creating a frame, and then loading it into a buffer which displays the frame.

A frame is what we call the "image" that is displayed at any given moment on the matrix. If an animation is a series of images, a frame is one of those images in the series.

In order to control the 12x8 LED matrix on the UNO R4 WiFi, you need a space in memory that's at least 96 bits in size. The library provides two ways to do this.

The first is simply to make a two-dimensional array of bytes like so:

```
1 byte frame[8][12] =  
2 { 0, 0, 1, 1, 0, 0,  
3 { 0, 1, 0, 0, 1, 0,  
4 { 0, 1, 0, 0, 0, 1,  
5 { 0, 0, 1, 0, 0, 0,  
6 { 0, 0, 0, 1, 0, 0,  
7 { 0, 0, 0, 0, 1, 0,  
8 { 0, 0, 0, 0, 0, 1,  
9 { 0, 0, 0, 0, 0, 0,  
10 };
```

This option is simple to understand, because you can see the image in the pattern of the array, and it is easy to edit in runtime. The ones in the array above form a heart, and that's the image you'd see on the screen.

To target an individual pixel you select its address and change the value, remember that you'll need to start counting at 0. So, the following line will target the third pixel from the top and the second from the left, then turn it on:

```
1 frame[2][1] = 1;  
2  
3 matrix.renderBitmap(f
```

This method takes more memory than is needed, however. Even though each LED needs only a single bit to store its state, you're using eight bits (a byte). The more memory-efficient method to store a frame is to use an array of 32-bit integers.

In this section we'll walk through the process and the concept of how you may create one of these frames yourself. Though we have developed a tool that can do this for you, so you can [click here](#) if you want to skip this exercise.

Here's the same heart in that form:

```
1 unsigned long frame[  
2     0x3184a444,  
3     0x42081100,  
4     0xa0040000  
5 };
```

An unsigned long variable holds 32 bits, and 96/32 is 3, so an unsigned long array is an efficient way to hold all the bits you need for the LED matrix.

But how do those hexadecimal values relate to the positions of the pixels? To find out, convert

binary values. Here's a code snippet that will do this:

```
1 for (int b = 0; b < 32; b++)  
2   Serial.println(fra  
3 }
```

This will print out all the bit values of the array. The output will look like this:

```
1 1100011000010010100100  
2 1000010000010000001000  
3 1010000000000100000000
```

This method doesn't show you all the bits, though. Each array element should have 32 bits. If you add zeros to show all 32 bits of each element, you get:

```
1 0011000110000100101001  
2 0100001000001000000100  
3 1010000000000100000000
```

Now divide it into groups of 12 bits and you've got the heart back:



```
1 001100011000
2 010010100100
3 010001000100
4 001000001000
5 000100010000
6 000010100000
7 000001000000
8 000000000000
```

Hint: You can see the heart easier if you highlight all "1"s on the page by pressing CTRL/command + F and search for "1".

If you've got several different frames, you can load and display them like this:

```
1 const uint32_t happy[4] = {
2     0x19819,
3     0x80000001,
4     0x81f8000
5 };
6
7 const uint32_t heart[4] = {
8     0x3184a444,
9     0x44042081,
10    0x100a0040
11 };
12
13 matrix.loadFrame(happy);
14 delay(500);
15
16 matrix.loadFrame(heart);
17 delay(500);
```

Testing It Out

Let's apply these concepts, with two basic sketches that display

First, let's create 3x32-bit integer frames and load them one by one.

Here's a sketch that will first load a smiley face on your matrix, and then change it to a heart.

```
1  #include "Arduino_LED_Matrix.h"
2
3  ArduinoLEDMatrix matrix;
4
5  void setup() {
6      Serial.begin(115200);
7      matrix.begin();
8  }
9
10 const uint32_t happyFace[] = {
11     0x19819,
12     0x80000001,
13     0x81f8000
14 };
15 const uint32_t heart[] = {
16     0x3184a444,
17     0x44042081,
18     0x100a0040
19 };
20
21 void loop(){
22     matrix.loadFrame(happyFace);
23     delay(500);
24
25     matrix.loadFrame(heart);
26     delay(500);
27 }
```

The sketch is pretty simple, and yet the outcome is very expressive and can help you easily indicate states of your projects.

Now let's change approach and create a bitmap that we change in runtime. This sketch includes

winks the left eye by turning off certain pixels.

```
1  #include "Arduino_L...
2
3  ArduinoLEDMatrix ma
4
5  void setup() {
6      Serial.begin(1152
7      matrix.begin();
8  }
9
10 uint8_t frame[8][12
11     { 0, 0, 0, 0, 0,
12     { 0, 0, 0, 0, 0,
13     { 0, 0, 0, 0, 0,
14     { 0, 0, 0, 0, 0,
15     { 0, 0, 0, 0, 0,
16     { 0, 0, 0, 0, 0,
17     { 0, 0, 0, 0, 0,
18     { 0, 0, 0, 0, 0,
19 };
20
21 void leftEye() {
22     //Left eye
23     frame[1][3] = 1;
24     frame[1][4] = 1;
25     frame[2][3] = 1;
26     frame[2][4] = 1;
27 }
28
```

Resources

The rest of this article is a collection of resources, such as functioning examples, a gallery of frames, and tools that can help you get started with the LED Matrix in different ways.

Scrolling Text

Example

The LED Matrix now supports printing characters via the [ArduinoGraphics](#) library. With it, you are able to:

Set a start location for the text via

```
matrix.beginText(x,y,  
0xFFFFF)
```

. The "0xFFFFF" represents the default color (red). As the **ArduinoGraphics** library supports other hardware with multiple colors, we need to specify it.

Print the text via

```
matrix.printText("This message is  
printed")
```

End the print and (optionally) specify scroll direction with

```
matrix.endText(direction)
```

```
SCROLL_LEFT,
```

```
SCROLL_RIGHT
```

 are

supported. Leave blank if no scroll is desired.

The example below simply prints out **"Hello World!"** on the matrix.

For this example to work, you need to install the [ArduinoGraphics](#) library



to enable character printing on the LED matrix using the library manager.

```
1 // To use ArduinoGr
2 #include "ArduinoGr
3 #include "Arduino_L
4
5 ArduinoLEDMatrix ma
6
7 void setup() {
8   Serial.begin(11520
9   matrix.begin();
10
11   matrix.beginDraw(
12   matrix.stroke(0xF
13   // add some statio
14   // will only show
15   const char text[]
16   matrix.textFont(F
17   matrix.beginText(
18   matrix.println(te
19   matrix.endText();
20
21   matrix.endDraw();
22
23   delay(2000);
24 }
25
26 void loop() {
27
28   // Make it scroll
```

Frame Gallery

We've designed a gallery of frames and animations that are included in the library! You may load and display them on your UNO R4 WiFi with the following code snippet:



```
1 #include "Arduino_LED_Matrix.h"
2
3 ArduinoLEDMatrix matrix;
4
5 void setup() {
6   Serial.begin(115200);
7   matrix.begin();
8
9 }
10
11 void loop() {
12   // Load and display the first frame
13   matrix.loadFrame(LED_MATRIX_UNO);
14 }
```

By changing the parameter in `matrix.loadFrame()` in the loop, you can choose among the available frames we designed.

The available frames are:

LEDMATRIX_BLUETOOTH

LEDMATRIX_BOOTLOADER_ON

LEDMATRIX_CHIP

LEDMATRIX_CLOUD_WIFI

LEDMATRIX_DANGER

LEDMATRIX_EMOJI_BASIC

LEDMATRIX_EMOJI_HAPPY

LEDMATRIX_EMOJI_SAD

LEDMATRIX_HEART_BIG

LEDMATRIX_HEART_SMALL

LEDMATRIX_LIKE

LEDMATRIX_MUSIC_NOTE

LEDMATRIX_RESISTOR

LEDMATRIX_UNO

Alternatively, play one of the animations on the LED matrix like this:

```
1 #include "Arduino_LED_Matrix.h"
2
3 // Create an instance of the class
4 ArduinoLEDMatrix matrix;
5
6 void setup() {
7   Serial.begin(115200);
8   // you can also load a sequence
9   matrix.loadSequence(LED_MATRIX_SEQUENCE_1);
10  matrix.begin();
11  matrix.play(true);
12 }
13
14 void loop() {
15 }
```

In this case, you change the parameter of `matrix.loadSequence()` in the setup to one of the available ones to display.

The available animations are:

LEDMATRIX_ANIMATION_STARTUP

LEDMATRIX_ANIMATION_TETRIS_INTRO

LEDMATRIX_ANIMATION_ATHLETA_TMEGA

LEDMATRIX_ANIMATION_LED_BLINK_HORIZONTAL

LEDMATRIX_ANIMATION_LED_BLINK_VERTICAL

LEDMATRIX_ANIMATION_ARROWS_COMPASS

LEDMATRIX_ANIMATION_AUDIO_WAVEFORM

LEDMATRIX_ANIMATION_BATTERY

LEDMATRIX_ANIMATION_BOUNCING_BALL

LEDMATRIX_ANIMATION_BUG

LEDMATRIX_ANIMATION_CHESS

`LEDMATRIX_ANIMATION_DOWNLOAD``LEDMATRIX_ANIMATION_DVD``LEDMATRIX_ANIMATION_HEARTBEAT_LINE``LEDMATRIX_ANIMATION_HEARTBEAT``LEDMATRIX_ANIMATION_INFINITY_LOOP_LOADER``LEDMATRIX_ANIMATION_LOAD_CLOCK``LEDMATRIX_ANIMATION_LOAD``LEDMATRIX_ANIMATION_LOAD_CLOCK``LEDMATRIX_ANIMATION_NOTIFICATION``LEDMATRIX_ANIMATION_OPEN_SOURCE``LEDMATRIX_ANIMATION_SPINNING_COIN``LEDMATRIX_ANIMATION_TETRIS``LEDMATRIX_ANIMATION_WIFI_SEARCH`

Animation Generation

We have developed a tool that is used to generate frames and animations to be rendered on the LED Matrix in your browser. This tool is part of [Arduino labs](#), and is therefore considered experimental software.

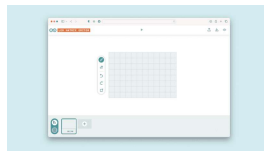
To use the tool you need to upload the following sketch, allowing the board to read serial inputs sent by the browser.

You can also find the sketch in

> LivePreview

```
1 #include "Arduino_LED_Matrix.h"
2
3 ArduinoLEDMatrix matrix(1, 1);
4
5 void setup() {
6     Serial.begin(115200);
7     matrix.begin();
8 }
9
10 uint32_t frame[] = {
11     0, 0, 0, 0xFFFF
12 };
13
14 void loop() {
15     if(Serial.available()) {
16         frame[0] = Serial.read();
17         frame[1] = Serial.read();
18         frame[2] = Serial.read();
19         matrix.loadFrame(frame);
20     }
21 }
```

[Click here](#) to go to the LED Matrix tool.



LED Matrix Editor

Once you've made your animations, you can export them from the tool in the format that lets you use them as [previously discussed](#).

You can find more tips on how to use this tool on [its site](#).

API

To write more advanced sketches on your own, you may use the full API of the library as found below.

Members	Descriptions
public ArduinoLEDMatrix ()	The main class for controlling the LED matrix.
public void autoscroll (int32_t interval_ms)	Sets the time in ms for each frame to be displayed.
public void begin ()	Start the LED matrix.
public void next ()	Manually move to the next frame in the sequence.
public void loadFrame (const uint32_t buffer[3])	Load a new single frame that is not in any sequence.
public void renderFrame (uint8_t frameNumber)	Render the loaded frame.
public void loadSequence (const uint32_t frames[][4])	Loads an animation sequence into the buffer but does not display it.
public	Start playing the

Members	Descriptions
<code>(bool loop = false)</code>	option to loop indefinitely or play once.
<code>public bool sequenced one</code> <code>()</code>	checks if the sequence has finished playing.

Members

`ArduinoLEDMatrix()`

Construct a new LED matrix object. This will be used to access the methods in the library.

```
1 ArduinoLEDMatrix LEDMa
```

`autoscroll()`

Enable autoscrolling through the frames in a sequence.

Parameters

`interval_ms` Sets the time in milliseconds that should be spent on a frame before switching to the next frame in the sequence.

`begin()`

Starts the LED matrix.

```
1 LEDMatrix.begin()
```

`next()`

Manually moves to the next frame in the sequence.

```
1 LEDMatrix.next()
```

`loadFrame()`

Loads a single frame that is not part of a sequence.

```
1 LEDMatrix.loadFrame(buffer)
```

Parameters

`buffer[3]` an array of three 32bit integers, where each bit represents an LED.

`renderFrame()`

Render a specific frame from a sequence.

```
1 LEDMatrix.renderFrame(frame)
```

Parameters

`frame` the frame to render

`loadSequence()`

Loads an animation sequence into the buffer but does not display it.

```
1 LEDMatrix.frames[][4]
```

Parameters

`frameNumber` Specifies which frame of the sequence should be rendered.

`play()`

Starts playing the loaded sequence.

```
1 LEDMatrix.play(state)
```

Parameters

`loop` true to enable looping the sequence, false to play once.

`sequenceDone()`

Check for if the sequence is finished playing or if the frame should be advanced another step.

Returns

`false` if the sequence is not finished, `true` if it is.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

